System Analysis Activities

Katerina Mangaroska week 36

01

4+1 views paradigm

4 + 1 views paradigm

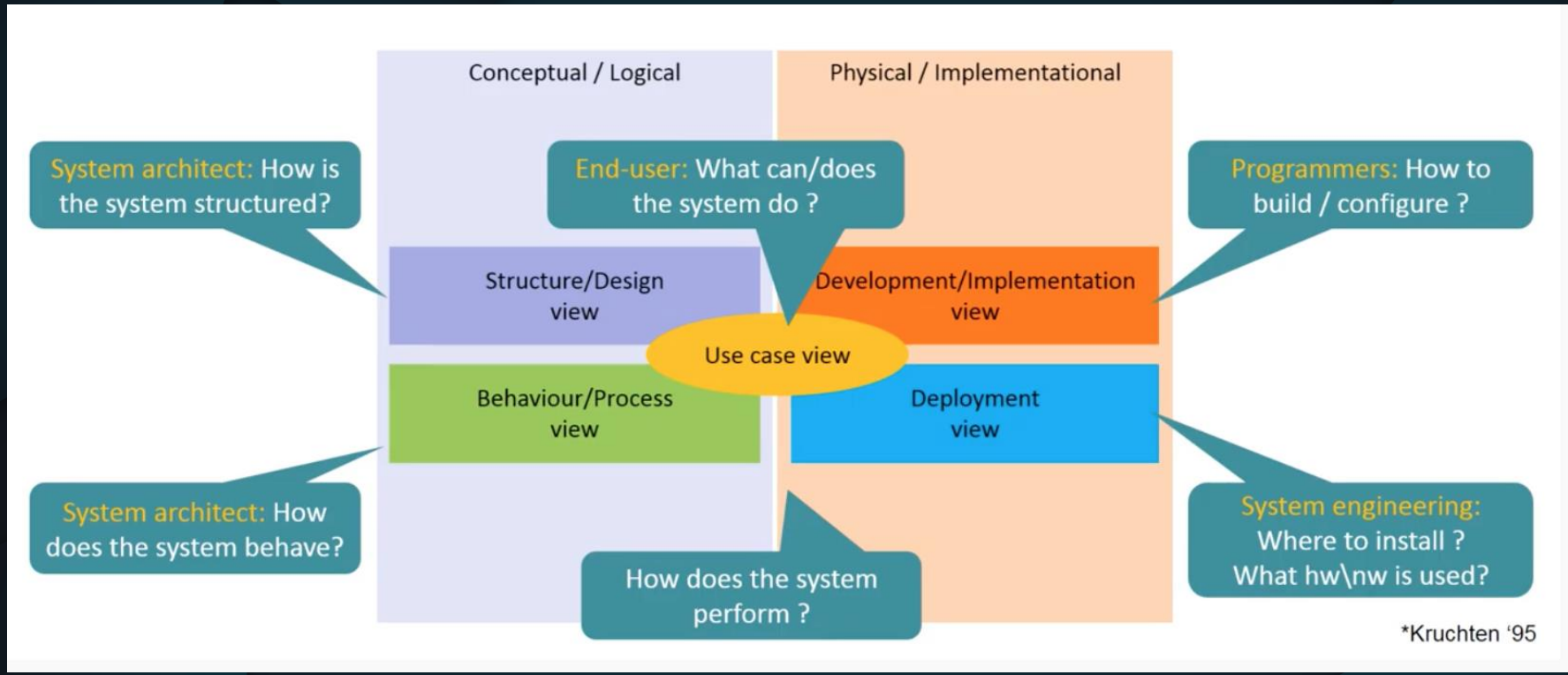
Systems are described from many viewpoints.

- ✓ view: a representation of a structure.
- ✓ structure: set of elements, as they exist in the software system;

An architectural view is a simplified description (abstraction) of a system.

- from a particular purpose (stakeholders and concerns)
- omitting entities that are not relevant

Architecture is represented by means of one or more architectural models that together provide a coherent description of the system architecture.


Conceptual / Logical
Physical / Implementational
System architect: How is the system structured?
End-user: What can/does the system do ?
Programmers: How to build / configure ?
Structure/Design view
Development/Implementation view
Use case view
Behaviour/Process view
Deployment view
System architect: How does the system behave?
How does the system perform ?
System engineering: Where to install ? What hw\nw is used?
*Kruchten '95

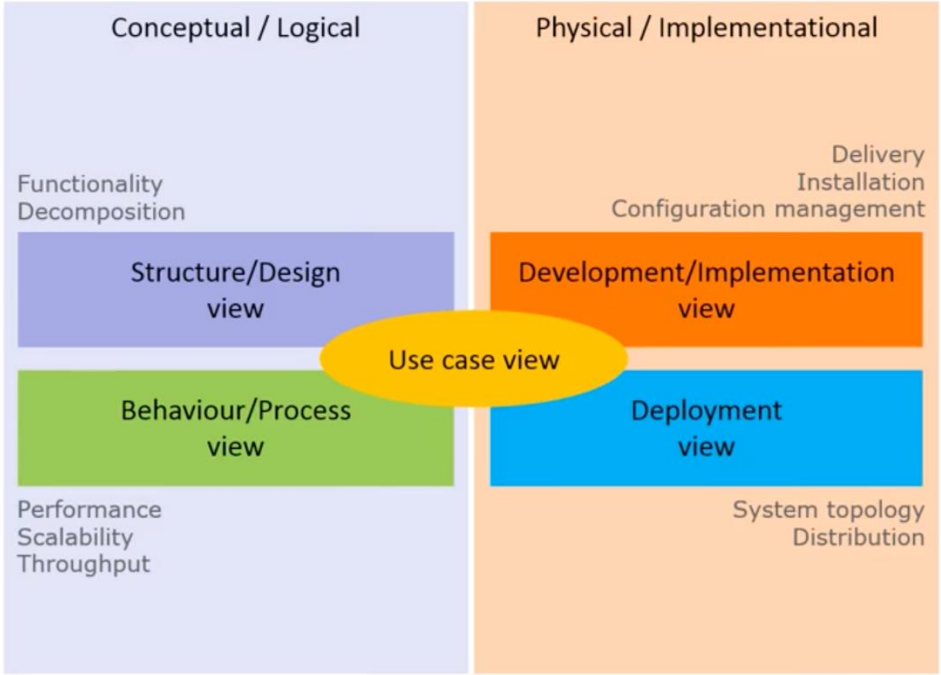
Component diagram
Class diagram

Sequence diagram
State diagram
Activity diagram

Mapping Components onto files

Deployment diagram

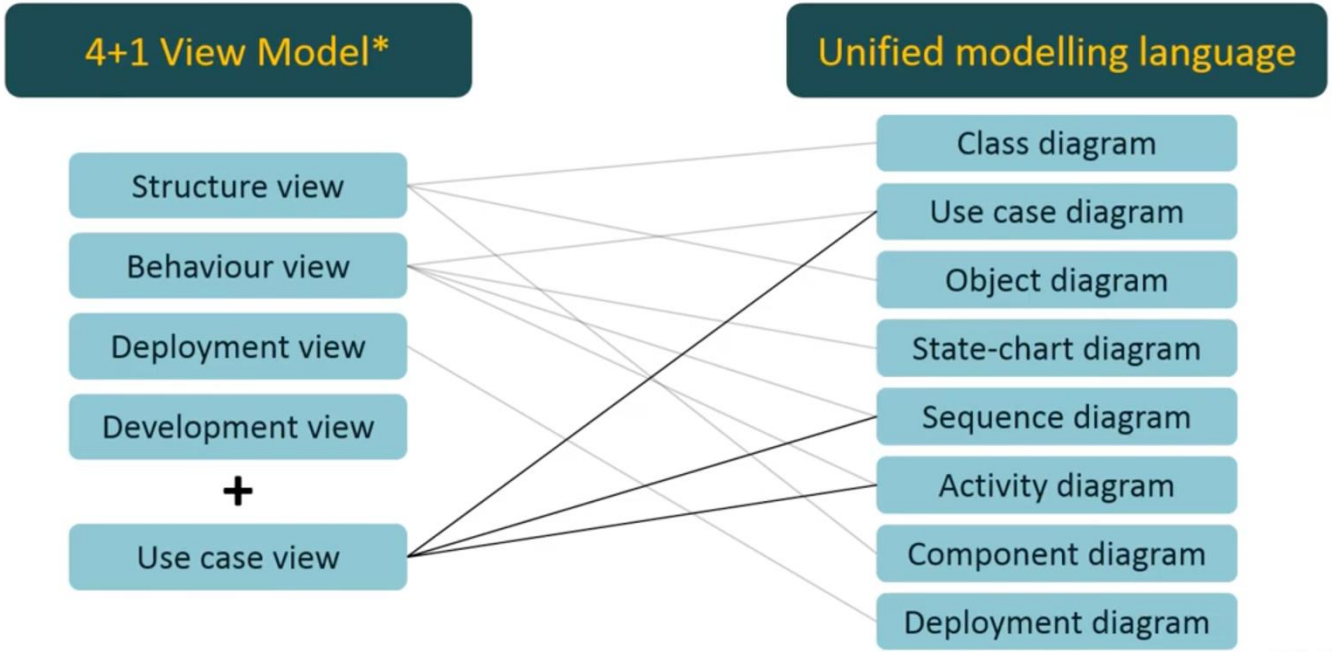
Use case diagram
Context diagram



*Kruchten '95

- Structure/ Design view - concerned with the functionality that the system provides to end-users.
- Behavior/Process view - concerned with the dynamic aspects of the system, explains the system processes and how object communicates.
- Development/Implementation view describes how code is organized during development, such as mapping components onto files.
- Deployment view concerns with the topology of software components on the physical layer, as well as the physical connection between these components.
- The use of a system is described through use cases. Use cases describe interactions between actors and the system itself.

Multiple diagram types per View



Multiple diagram types may be used in one view

*Philippe Kruchten

Requirements modeling

Objective: understanding – users' needs, how the business processes are carried out, how the system will support those business processes.

Object-oriented analysis - set of analysis activities to understand needs and processes.

- discovery activities (facts finding activities)
- modeling and documenting system requirements

Requirements

Business requirements - state the problem and the business objective to be achieved.

- increased revenue/throughput/customer reach,
- reduced expenses/errors,
- improved customer service, etc.

User (stakeholder) requirements - serve as a bridge between generalized business requirements and specific solution requirements.

- ability to create various reports, view order history and status, manage customer databases

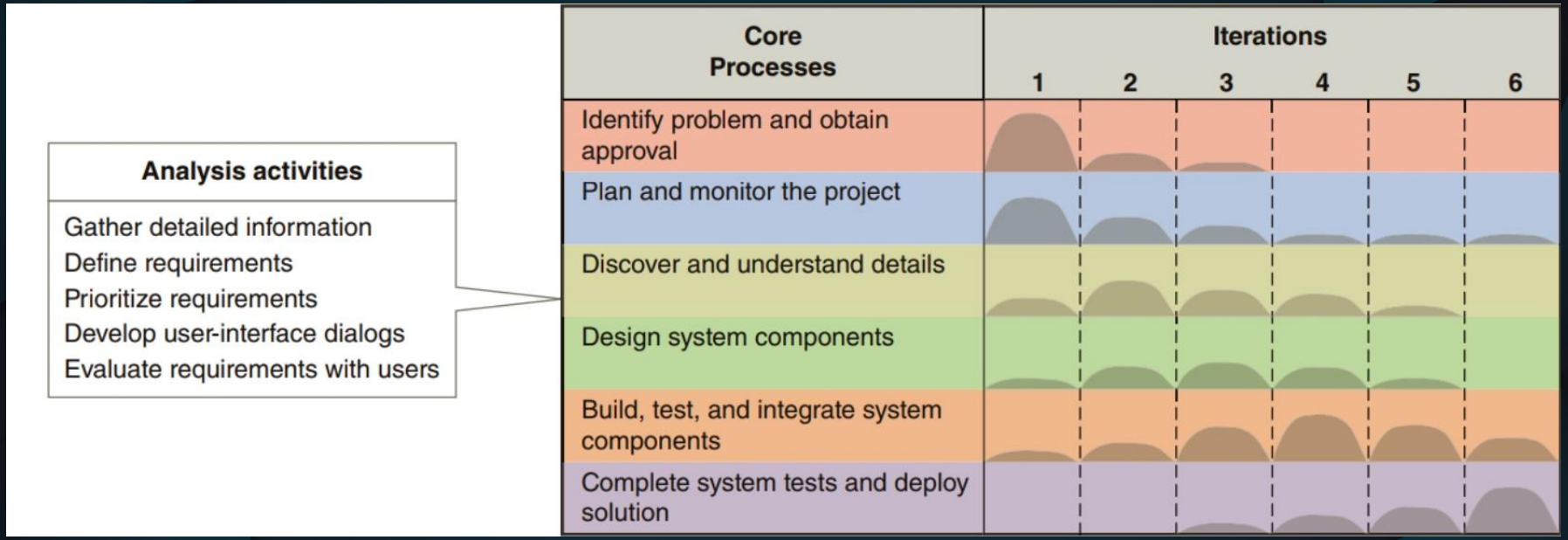
Solution (software) requirements - describe specific characteristics that a product must have to meet the needs of the stakeholders and the business itself.

Object-oriented analysis

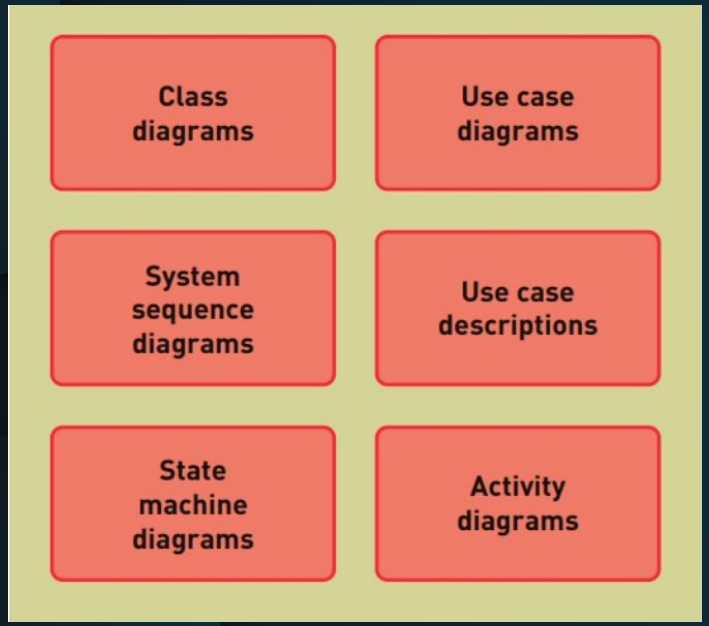
is the procedure of identifying software engineering requirements and developing software specifications in terms of a software system's object model, which comprises of interacting objects.

difference: requirements are organized around objects, which integrate both data and functions. They are modelled after real-world objects that the system interacts with.

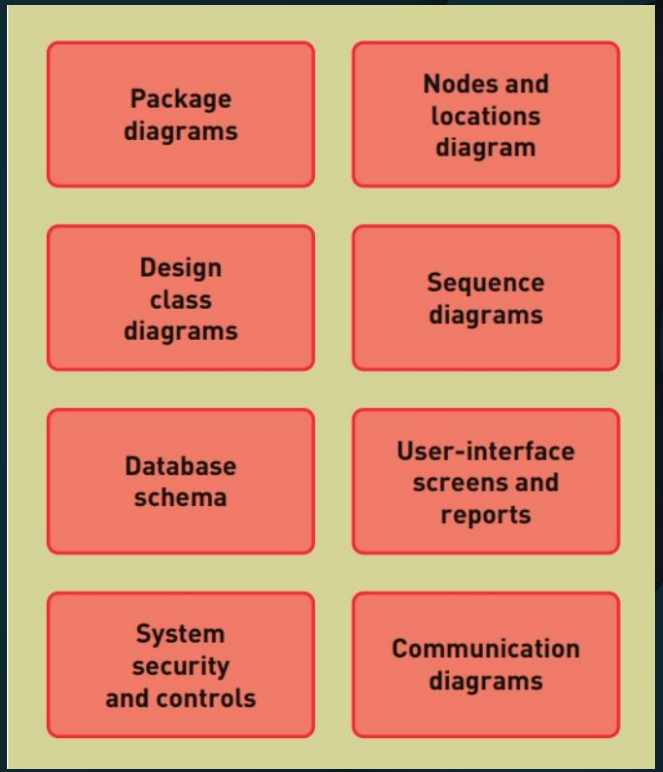
System analysis activities



analysis models



design models



02

Software Requirements

Software requirements

System requirements are all the activities the new system must perform or support and the constraints that the new system must meet.

Functional requirements are the activities that the system must perform.

e.g., developing a payroll system - calculate commission amounts, calculate payroll taxes, report tax deductions to the IRS.

Non-functional requirements (aka quality attributes) are characteristics of the system other than those activities it must perform or support.

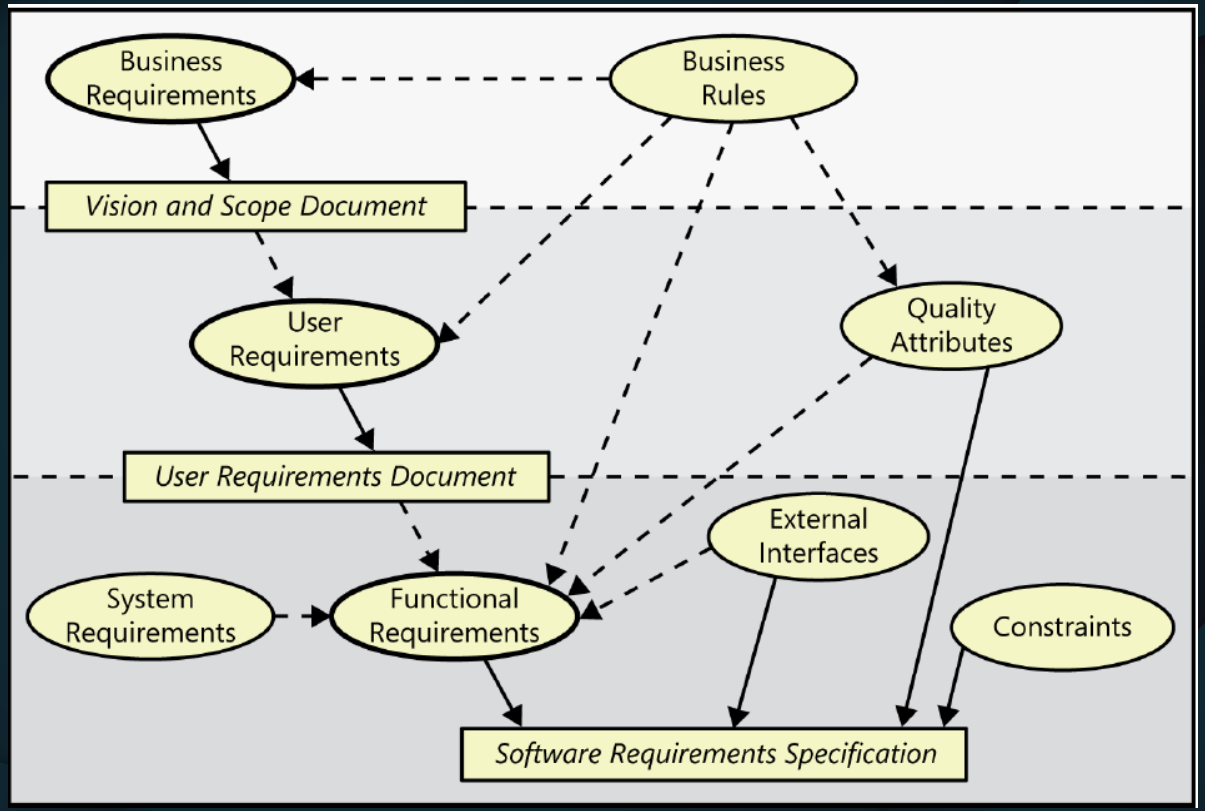
- ✓ functional requirements define what a system is supposed **to *do*** and non-functional requirements define how a system is supposed **to *be***.

Software requirements specification

Top tier – the high-level business requirements dictating the goal behind the product.

Middle tier – user requirements that picture the end-users and their needs.

Bottom tier – SRSs that specify the product's functionality in tech terms.



Software requirements specification

- is a technical document that describes future software functions and capabilities, its characteristics, design, and implementation constraints for the development team.

Purpose. Definitions, system overview, and background.

Overall description. Assumptions, constraints, business rules, and product vision.

Specific requirements. System attributes, functional requirements, and database requirements.

Functional requirements

Functional requirements include *features and functions* that allow the system to perform as intended. They are composed of:

- if/then behaviours,
- description of the system's workflow,
- logic behind data handling, and
- data inputs and corresponding outputs.



Functional requirements describe the system's behaviour in response to the input.

Functional requirements

Functional requirements are usually in the form of "system shall **do** <requirement>".

Examples:

The system must send a descriptive error response if an issue arises during users' message handling.

- The requested URL /page.html was not found on this server.
- Apache/2.2.3 (Unix) mod_ssl/2.2.3 OpenSSL/0.9.7g DAV/2 PHP/5.1.2 Server at localhost Port 80

Functional requirements

Functional requirements are usually in the form of "system shall **do** <requirement>".

Examples:

The system must send a descriptive error response if an issue arises during users' message handling.

The system shall allow operators to create new users.

The system shall allow administrators to see active airplanes on a map.

The system will send a confirmation email when a new user account is created.

Are these functional requirements? If yes, are they well written or not?

Examples to discuss:

1. As a user I want to converse via an integrated email inbox to keep in touch with the administrator and the attendees about changes/cancelations of events.
2. Dashboard for device, see details from individual devices.
3. Allow users to create custom rules that will trigger different events based on conditions.
4. The system must attach a geo location to the reported picture, if the user has agreed to it.
5. The system must be available in both Norwegian and English.
6. The system must display sensors in both a list format and a map format.
7. The system must function equally well on all standard screen formats.

Are these functional requirements? If yes, are they well written or not?

Examples to discuss:

8.The website shall report online volume statistics every four hours and hourly during peak periods.

9. The system must provide log on security at the operating system level and at the application level.

10. The contact management system shall generate a daily reminder list for all sales representatives.

11. The inventory system shall produce a daily report showing the part number, description, quantity on hand, quantity allocated, quantity available, and unit cost of all items sorted by part number.

12. The system must maintain separate levels of security for users and the system administrator.

Are these functional requirements? If yes, are they well written or not?

9. The system must provide log on security at the operating system level and at the application level.

can be seen as both a functional and a non-functional requirement, depending on the context and the specifics of the requirement:

As a Functional Requirement: If the requirement specifically dictates what the system should do regarding log on security (e.g., implementing password checks, biometric scans, two-factor authentication), it acts as a functional requirement. It describes specific functionalities that the system must provide to achieve security during the log-in process.

As a Non-Functional Requirement (Security): If the emphasis is on the quality of the security (e.g., the level of security, such as "high-security" log-ins, encryption standards, compliance with certain security protocols), it could be considered a non-functional requirement. It describes the quality attributes or security standards the system must meet, rather than specific functions.

Non-functional requirements

Non-functional requirements (NFR) define system behavior, features, and general characteristics that affect the user experience. NFR are statements of operational system constraints.

- They represent a set of standards used to judge the specific operation of a system. Example, how fast does the website load?

  Although there are hundreds of NFRs, the most common types are:

  - performance and scalability;
  - portability and compatibility;
  - reliability, availability, maintainability;
  - security;
  - localization; and
  - usability.

Non-functional requirements

Non-functional requirements are in the form of "system shall **be** <requirement>”

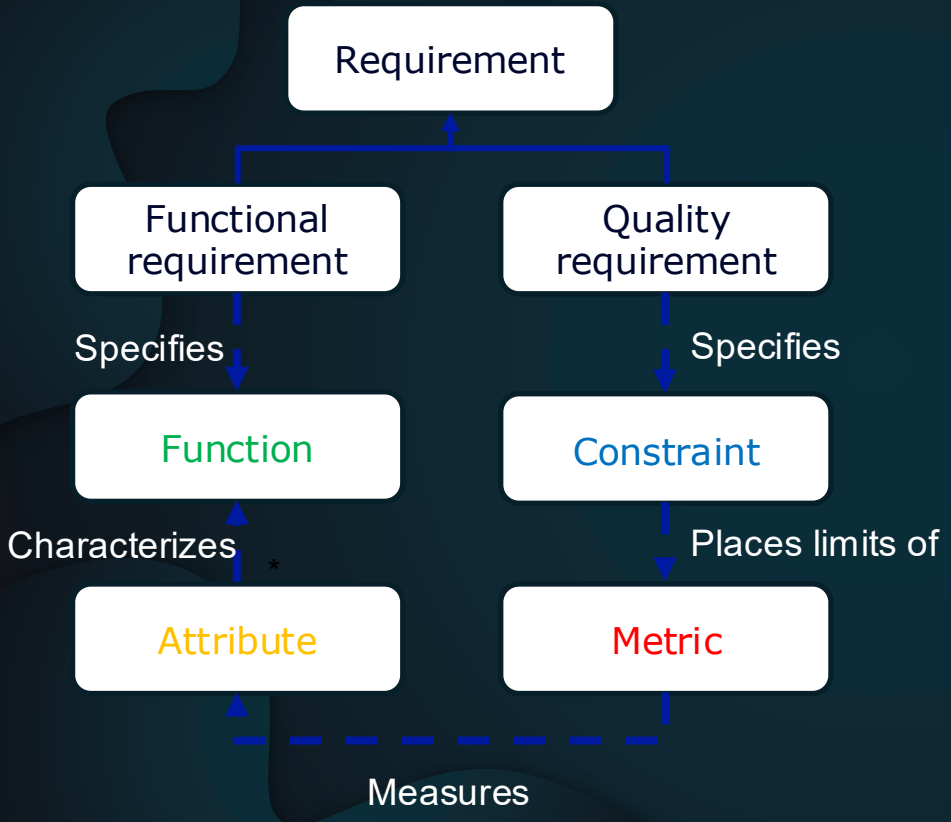
Examples:

(performance) The processing time should not exceed 5 seconds given a service inventory of 1000 services.

(reliability) The system must restart within 1 minute when it crashes.

(performance) The website pages should load in 3 seconds with the total number of simultaneous users <5 thousand.

Functionality and qualities are related



- [functionality] Through the portal, amount of customers can browse the catalog of items on sale
- [performance] At least 10,000 customers should be able to access the portal at the same time

Non-functional requirements

FURPS is an acronym that stands for *f*unctional, *u*sability, *r*eliability, *p*erformance, and *s*ecurity.

Usability requirements describe operational characteristics related to users, such as the user interface, related work procedures, online help, and documentation.

Reliability requirements describe the dependability of a system; how it detects and recovers from service outages and incorrect processing.

Performance requirements describe operational characteristics related to measures of workload, such as throughput and response time.

Security requirements describe how access to the application will be controlled and how data will be protected during storage and transmission.

Usability requirements

- How easy is it for a customer to use the system?

Example: *The error rate of users submitting their payment details at the checkout page mustn't exceed 10 percent.*

Five dimensions to evaluate usability:

1. **Learnability.** How fast is it for users to complete the main actions once they see the interface?
2. **Efficiency.** How quickly users can reach their goals?
3. **Memorability.** Can users return to the interface after some time and start efficiently working with it right away?
4. **Errors.** How often do users make mistakes?
5. **Satisfaction.** Is the design pleasant to use?

Reliability requirements

- How likely the system or its element would run without a failure for a given period of time under predefined conditions?

Example: Under normal usage conditions, there's an 85 percent chance that the system won't experience critical failure for a month.

Three ways to measure reliability:

1. Probability percentage, time;
2. The number of critical failures, time; and
3. Mean time between failures.

Performance requirements

Performance defines how fast a software system or its particular piece responds to certain users' actions under certain workload. In most cases, this metric explains how much a user must wait before the target operation happens (the page renders, a transaction is processed, etc.) given the overall number of users at the moment.

- How fast does the system return results?

Example: *The landing page supporting 5 thousand users per hour must provide 6 seconds or less response time in a Chrome desktop browser, including the rendering of text and images, over an LTE connection (type of 4G network).*

Three main metrics for a response time:

- *0.1 second* – the limit after which the system reaction doesn't seem instantaneous;
- *1 second* – when user will notice the delay, but without interrupted flow of thought;
- *10 seconds* – when user attention is completely lost.

Security requirements

- How are the system and its data protected against attacks?

Example: The payment processing gateway must be PCI DSS compliant (Payment Card Industry Data Security Standard).

Expand non-functional requirements to functional ones. They can include, say, a comprehensive authorization and authentication scheme for each system actor. Also, the system is supposed to introduce constraints on who can generate, view, duplicate, edit, or delete the data.

Consider standards that you rely on. If your system must be compliant with some security standards or regulations, the non-functional section is the best place for them.

Are these non-functional requirements? If yes, are they well written or not?

Examples to discuss:

availability

1. The solution should be able to maintain more events than it currently does.

scalability

2. The application must allow users to stay signed in over a longer time period.

security

3. The system must be scalable to meet the growing number of internet users.

reliability

4. Database should follow GDPR.

5. In case data is lost, it should be easy to recover by taking frequent backups of the DB.

modifiability

6. The system needs to be easy to modify.

reliability

7. A routine to make backup(s) of the database should be established to ensure that data may be recovered in case of failure.

Are these non-functional requirements? Well written or not?

Examples to discuss:

supportability

8. Considering its potentially long lifespan, the system should be easy to update for developers in future.

scalability?

9. The site should be scalable. The site will fit every device in the form of size.

design constraints

10. Data entry screens must be uniform, except for background color, which can be changed by the user.

scalability

11. The system must support 25 users online simultaneously.

performance

12. Response time must not exceed four seconds.

compatibility

13. The system should work on Windows and Mac platforms.

Non-functional requirements

Requirement categories	FURPS + categories	Example requirements
Functional	Functions	Business rules and processes
Nonfunctional	Usability Reliability Performance Security	User interface, ease of use Failure rate, recovery methods Response time, throughput Access controls, encryption

© Cengage Learning®

Scalability requirements

Scalability assesses the highest workloads under which the system will still meet the performance requirements.

- How much will the (this) performance change with higher workloads?

Example: The database must be able to an annual growth rate of 20%, with no degradation in database performance.

Physical. Specifies the availability of the factors required in terms of the physical components of the process (i.e., data storage, network bandwidth, hardware, etc.) to qualify as sustainable.

Intangible. Refers to the inherent ability to support non-physical growth. Business growth is critical to maintaining the market stake and competitiveness. (e.g., new products)

Availability requirements

Availability describes how likely the system is accessible for a user at a given point in time.

- How much time is it available to users against downtimes?

Example: The web dashboard must be available to US users, 98 percent of the time every month during business hours EST.

Example: Less than 20 seconds shall be needed to restart the system after a failure 95% of the time.

Localization requirements

Localization defines how well a system or its element falls in line with the context of the local market-to-be. The context includes local languages, laws, currencies, cultures, spellings, and other aspects. The more a product sticks with it, the more success it should have with a particular target audience.

- Does the system match local specifics?

Example: The date format must be as follows: month.date.year. (US)

Example: The system must be available in both Norwegian and English language.

Portability requirements

Portability defines how a system or its element can be launched on one environment or another.

- Which hardware, operating systems, browsers, and their versions does the software run on?

Example: *No more than 5% of the system implementation shall be specific to the operating system.*

Can be measured as:

Number of targeted platforms (hardware, OS)

Proportion of platform specific components or functionality

Mean time to port to a different platform

Compatibility requirements

Compatibility defines how a system can co-exist with another system in the same environment. For instance, software installed on an operating system must be compatible with its firewall or antivirus protection.

- Does it conflict with other applications and processes within particular environments?

Example: *The iOS application must support iPhone devices running on the following OS versions: 3.6 and 4.3.*

Maintainability requirements

Maintainability defines the time required for a solution or its component to be fixed, changed to increase performance or other qualities, or adapted to a changing environment. E.g., extension with new functionalities, deleting unwanted capabilities.

- How easy is to repair?

Example: The cyclomatic complexity of the code must not exceed 7.

Example: No method in any object may exceed 200 lines of code.

*CYC is a software metric used to determine the complexity of a program. It is a count of the number of decisions in the source code.

Non-functional requirements FURPS +

Design constraints describe restrictions to which the hardware and software must adhere.

e.g., Android operating system, consume no more than 30 MB of flash memory storage, consume no more than 10 MB of system memory while running.

Implementation requirements describe constraints such as required programming languages.

Interface requirements describe interactions among systems.

e.g., must generate data in XML format, supply data directly to stock exchanges and bond rating agencies

Physical requirements describe such characteristics of hardware as size, weight, power consumption, and operating conditions.

e.g., operating for 48 hours on a fully charged 1200 milliwatt lithium ion battery

Supportability requirements describe how a system is installed, configured, monitored, and updated.

Making the NFR testable

Performance: A REST service call must respond with 500 milliseconds at the 95th percentile.

Scalability: A REST service must be able to support 10 requests a second and still meet its response time requirement.

Utilization: The application server shall not exceed 50% CPU consumption.

What you need to test. *A REST service must be able to support 10 requests a second, and must respond with 500 milliseconds at the 95th percentile, and the application server shall not exceed 50% CPU consumption.*

Making the NFR testable

It’s an import point that non-functional requirements do not need to be **tested in isolation**, it is true to say that some do but for the most part they cannot, and you need to combine them to make them testable.

For the most part you are going to have to **automate these tests.** Do not **over-engineer** your scripts or tests. **However,** some non-functional testing requires a **manual approach**, those supporting **Recoverability** and **Disaster Recovery** in particular.

Further reading: https://octoperf.com/blog/

FUNCTIONAL vs NONFUNCTIONAL REQUIREMENTS

	Functional requirements	Nonfunctional requirements
Objective	Describe what the product does	Describe how the product works
End result	Define product features	Define product properties
Focus	Focus on user requirements	Focus on user expectations
Documentation	Captured in use case	Captured as a quality attribute
Essentiality	They are mandatory	They are not mandatory, but desirable
Origin type	Usually defined by user	Usually defined by developers or other tech experts
Testing	Component, API, UI testing, etc. Tested before nonfunctional testing	Performance, usability, security testing, etc. Tested after functional testing
Types	External interface, authentication, authorization levels, business rules, etc.	Usability, reliability, scalability, performance, etc.

03
User stories

User stories and use cases

Identifying user stories and use cases is a key task when defining functional requirements because these form the basis for the list of functions the system needs to carry out.

They focus on the goals of the user, and they show the list of functions at the appropriate level of detail.

Virtually all recent approaches to system development begin the **requirements modeling** activity with the **concept of a user story or a use case**.

User stories are favored by highly Agile system development methodologies, and they are turned over to the programmer analyst much earlier than use cases are. They are a basic concept in Agile development because they focus on simplicity, value added, and user collaboration.

User stories and use cases

A **user story** describes a goal the user has when using the system.

- document the functional requirements less formally than traditional requirements modeling by focusing on *who*, *what*, and *why* for each function.

Examples:

As a user I should be able to see a list of services in the service inventory.

As a user, I should be able to see how many cars that are driving in Trondheim on a map are in a particular area of the city, to know what areas to avoid.

User stories

Acceptance criteria: indicate the features that must be present for the user to be satisfied with the resulting implementation.

They focus on functionality, not on features or user-interface design.

ID	Priority	Description	Story Points	Acceptance Criteria
2	1	As a user, I want to be able to log in, so that I can access the system.	17	• If I am a user of the system I can log in to the system. • I can log in using my email and password, registered in the system. • The system notifies me if I have entered invalid login credentials. • If invalid login credentials are supplied I am prompted to retry logging in. • If the login is successful I am redirected to the landingpage.

Proper acceptance criteria

ID	Priority	Description	Story Points	Acceptance Criteria
1	-	As an admin, I should be able to access logging, so that I'm able to debug fatal errors that may occur.	10	• If I am an admin I can view a logfile. • The logfile is located on the server. • The logfile contains relevant info regarding errors that have occurred.
2	1	As a user, I want to be able to log in, so that I can access the system.	17	• If I am a user of the system I can log in to the system. • I can log in using my email and password, registered in the system. • The system notifies me if I have entered invalid login credentials. • If invalid login credentials are supplied I am prompted to retry logging in. • If the login is successful I am redirected to the landingpage.

Could be improved

Table 7: Implemented User Stories

ID	Priority	User Story	Estimate	Acceptance Criteria
1	XL	As a user, I want to see a map of the contract area	M	Given I'm a user when I'm looking at the website, then I see a map over the contract area.
2	XL	As a user, I want the ability to upload R20 documents to the system	XL	Given I'm a user when I upload an R20 document, then I see the data in the system
3	XL	As a user, I want to see discrepancies on the map	S	Given I'm a user when I'm looking at the map, then I see discrepancies

Further reading

Testing blog - https://octoperf.com/blog/

System development - https://courses.lumenlearning.com/sanjacinto-computerapps/chapter/reading-2/

Types of requirements - https://www.altexsoft.com/blog/functional-and-non-functional-requirements-specification-and-types/

Does anyone have any questions?

THANK YOU!

katerina.mangaroska@usn.no